# LEGACY CODE MODERNISATION USING DIFFERENTIAL VALIDATION

**PROJECT SYNOPSIS**

OF MINI PROJECT

Submitted in partial fulfilment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**

in

Computer Science and Engineering

SUBMITTED BY

[Student Name(s)]

[Roll Number(s)]

[Month, Year]

**BIRLA INSTITUTE OF APPLIED SCIENCES, BHIMTAL**

District Nainital, Uttarakhand - 263136

(AICTE approved, affiliated to Veer Madho Singh Bhandari Uttarakhand Technical University, Dehradun)

| Submitted To | Submitted By |
|---|---|
| [Mentor Name and Designation] | [Student Name(s)] B.Tech CSE, VII Semester |
| Mentor Signature: _____ | Student Signature(s): _____ |

## Project Particulars

| Particular | Details |
|---|---|
| Project title | Legacy Code Modernisation Using Differential Validation |
| Student name(s) and roll number(s) | [Student Name(s)] / [Roll Number(s)] |
| Programme and semester | B.Tech Computer Science and Engineering, VII Semester |
| Project guide | [Mentor Name and Designation] |
| Academic session | [Academic Session] |
| Submission month and year | [Month, Year] |
| Project area | Software engineering, legacy modernisation and programme testing |

## Table of Contents


| Section | Page |
|---|---|
| Abstract | 3 |
| 1 Introduction | 4 |
| 2 Literature Survey | 5 |
| 3 Problem Formulation | 6 |
| 4 Objectives | 6 |
| 5 Methodology and Planning of Work | 7 |
| 6 Activity Chart | 8 |
| References | 9 |

## Abstract

Legacy COBOL programs can be difficult to modernise without changing established file formats, numeric behaviour or error handling. This project develops a local COBOL-to-Python translation workflow supported by differential validation. The original program supplies reference observations, while a local language model proposes a standalone Python candidate. Both implementations receive identical inputs in isolated Linux containers. Standard output, standard error, output-file presence and contents, and ordinary exit codes are compared exactly. Independent cases are generated from reviewed constraints, frozen and executed twice against COBOL before candidate validation. A bounded loop permits one generation and four repairs. The current prototype passes 74 automated tests and detects controlled byte, length, file-presence and exit-code differences. Its real 27B-model generation request timed out, so successful model translation remains unverified. Protected held-out evaluation is planned. The intended outcome is reproducible, auditable evidence of compatibility within a declared input domain, rather than universal equivalence.

Keywords: COBOL modernisation; differential testing; code translation; local language models; sandbox execution; regression testing.

# 1 Introduction

Legacy programs encode business rules together with assumptions about record layouts, decimal arithmetic, file handling and failure behaviour. Rewriting such programs in a contemporary language can improve maintainability, but an apparently reasonable translation may alter an externally visible result. GnuCOBOL provides an executable reference environment for the supported COBOL subset [1]. The migration task therefore requires evidence that a candidate preserves the behaviour of the original program under controlled conditions.

This project develops Legacy Diff, a local command-line workflow for translating standalone batch COBOL programs into Python. A reviewed manifest declares source files, fixtures, toolchains, outputs, resource limits and input constraints. The original program and each candidate execute in separate Docker workspaces with networking disabled. Differential testing compares implementations on common inputs [2]; here, comparison includes exact output bytes, file presence and ordinary exit codes. Whitespace and line endings are preserved because downstream systems may depend on them.

Initial support covers deterministic file-based batch programs and constrained fixed-width inputs. Database access, CICS, external program calls and active network dependencies are excluded. Compiler-based translation and neural transcompilation already address related migration tasks. This project focuses on a small, locally reproducible workflow that connects generation, stable reference observations, bounded repairs and retained evidence before any acceptance decision.

Table 1 Current implementation evidence

| Component | Present status |
|---|---|
| Manifest and readiness checks | Implemented |
| Isolated comparison and frozen suites | Implemented and verified with controlled Docker fixtures |
| Generation and repair orchestration | Implemented; deterministic response tests pass |
| Real local model translation | Request timed out; no successful candidate demonstrated |
| Held-out acceptance | Planned; no candidate accepted |

## 2 Literature Survey

Roziere et al. introduced TransCoder, an unsupervised neural transcompiler evaluated on functions across C++, Java and Python [3]. The work demonstrates the potential of learning translation from monolingual code, while its execution-based evaluation is more relevant to this project than similarity between source strings. Its results do not establish correctness for complete COBOL batch programs, record layouts or runtime-specific error behaviour.

Yang et al. developed Csmith to generate C programs with defined behaviour and expose compiler disagreements [4]. The study supports independent test generation and careful control of language semantics. Legacy Diff applies these principles to input records rather than generated C programs. The challenge is to distinguish a translation mismatch from undefined inputs, unstable reference behaviour or failures of the execution environment.

Roziere et al. subsequently used automated unit tests to filter incorrect translations during self-training [5]. This connects executable feedback with code translation, but passing available tests remains conditional on their coverage. The proposed repair loop will therefore use visible regression failures, while a separate held-out stage will assess cases that did not guide generation or repair.

Chen et al. evaluated language models for code generation using functional tests in HumanEval [6]. This provides a useful precedent for measuring execution outcomes rather than accepting plausible-looking source. HumanEval concerns function-level programming tasks; a COBOL migration workflow must additionally observe file presence, byte formatting, process termination and compatibility with a pinned runtime.

The Python decimal module documents explicit precision and rounding controls [7]. These facilities may help preserve COBOL arithmetic, although selecting Decimal alone does not reproduce every field-width, truncation or rounding rule. Docker documents container memory and other resource controls [8], which support repeatable execution but do not constitute proof of complete isolation. Ollama exposes a local chat interface for model requests [9]; service availability and installed model identity are distinct from successful inference on available hardware.

The reference runtime follows GnuCOBOL documentation [1], while the comparison strategy follows differential testing [2]. Together, these sources support a workflow in which a model proposes code and an independent validator measures its behaviour. The practical gap addressed here is the integration of reviewed input constraints, frozen and stable reference observations, exact file-and-stream comparison, bounded repair and retained evidence for a clearly limited COBOL-to-Python scope. No claim is made that the individual techniques are new or that agreement on a finite suite proves equivalence for every input.

## 3 Problem Formulation

The problem is to translate a supported standalone COBOL batch program into Python while preserving its observable behaviour over a declared input domain. Let R denote the reference, C the candidate, and O(P, x) the completed observation of program P on input x under the recorded environment. For a frozen suite S, regression agreement requires O(R, x) = O(C, x) for every x in S. Each observation includes stdout bytes, stderr bytes, ordinary exit code, and declared output-file names, presence and contents.

A syntactically valid translation may still change padding, end-of-file handling, error codes or numeric results. Manual inspection alone does not provide repeatable evidence for these details. Developers maintaining batch processing and educational migration prototypes need a way to identify differences, reproduce failures and understand the limits of a compatibility claim. Incomplete captures, unstable reference runs and resource failures must remain inconclusive. The proposed work preserves recorded reference behaviour, including known quirks, while keeping business-rule correction outside the translation task.

## 4 Objectives

1. Design a reviewed execution manifest that identifies sources, inputs, outputs, toolchain versions and resource limits, and rejects unsupported configurations.

2. Implement isolated reference and candidate execution with exact comparison of streams, declared files and ordinary exit codes, verified using controlled mismatch fixtures.

3. Develop reproducible fixture, boundary and seeded-random case construction, with frozen hashes and two reference executions for every case.

4. Integrate a replaceable local Ollama adapter and evaluate one initial generation plus at most four repairs, retaining prompts, candidates, progress and stop reasons.

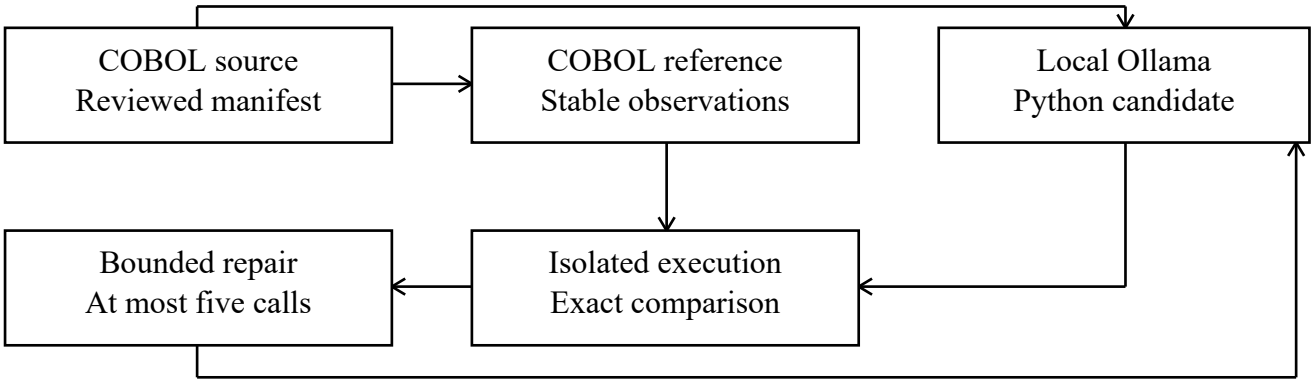
5. Evaluate compatibility on independently constructed regression and planned held-out cases, reporting complete results and keeping incomplete or failed runs unaccepted.

## 5 Methodology and Planning of Work

The team will study COBOL file and arithmetic behaviour and select approximately five to ten small, openly licensed batch programs. Selection will require available source, reproducible compilation, deterministic execution and reviewed inputs; unsupported database, CICS, external-call and network features will be excluded. Programmes will exercise report generation, fixed-width records, numeric boundaries, empty inputs and error paths.

For each program, the team will pin GnuCOBOL and Python images and declare execution conditions in a manifest. Fixtures, boundary cases and seeded random cases will be constructed independently of candidates. Every reference case will run twice before observations are frozen. Local Ollama will propose standalone Python, which will undergo artifact and syntax screening and Docker execution. Regression mismatches will guide at most four repairs; repeated candidates or failures will stop early.

Evaluation will record compilation success, complete case agreement, mismatch categories, reference stability, attempt counts, execution duration and resource failures. Hand-written matching and deliberately incorrect candidates will serve as harness controls. Planned held-out cases will remain outside repair feedback. Only public or synthetic data will be used, with source licences retained; private business records will not be uploaded. Current results will be reported separately from planned experiments and unresolved local-model resource limitations.




Regression pass -> awaiting acceptance -> held-out evaluation (planned)

Figure 1 Proposed generation and differential validation workflow

The validator owns the frozen suite and expected observations. The model returns candidate text only; candidate containers receive no reference executable or expected-data mount. Static screening supplements this boundary but cannot prove the absence of obfuscated delegation or precomputed-answer substitution.

## 6 Activity Chart

Table 2 Proposed semester work plan

| Period | Activity | Responsibility | Expected deliverable |
|---|---|---|---|
| Weeks 1-2 | Survey literature and review scope and constraints | [Team Member(s)] | Approved synopsis and execution contract |
| Weeks 3-4 | Implement manifest checks and isolated comparison | [Team Member(s)] | Runnable checker and mismatch fixtures |
| Weeks 5-6 | Construct independent suites and verify reference stability | [Team Member(s)] | Frozen cases, observations and integrity reports |
| Weeks 7-8 | Integrate local generation and bounded repair | [Team Member(s)] | Retained prompts, candidates and stop diagnostics |
| Weeks 9-10 | Implement held-out acceptance and expand programme fixtures | [Team Member(s)] | Separate acceptance evidence and completeness gates |
| Weeks 11-12 | Evaluate results, reproduce failures and prepare demonstration | [Team Member(s)] | Evaluation tables, project report and presentation |

Implementation progress

Manifest checking, isolated comparison, frozen regression suites and generation/repair orchestration have been implemented. The current automated suite contains 74 passing tests. Controlled Docker fixtures verify byte, length, file-presence and exit-code differences. These tests validate the harness; they do not measure translation accuracy across a COBOL benchmark.

The first real qwen3.8:27b generation request timed out after the configured wait and remained unaccepted. The remaining work includes feasible local-model inference, a broader licensed program collection, protected held-out acceptance, inspection/replay support and evaluation of complete generated candidates. No percentage accuracy, speed-up or successful industrial migration is claimed.

## References


[1] GnuCOBOL Project, "GnuCOBOL Manual." [Online]. Available: https://gnucobol.sourceforge.io/doc/gnucobol.html. Accessed: Oct. 10, 2026.

[2] W. M. McKeeman, "Differential testing for software," Digital Technical Journal, vol. 10, no. 1, pp. 100-107, 1998. [Online]. Available: https://vmssoftware.com/docs/dtj-v10-01-1998.pdf.

[3] B. Roziere, M.-A. Lachaux, L. Chanussot, and G. Lample, "Unsupervised translation of programming languages," in Advances in Neural Information Processing Systems, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2006.03511.

[4] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation, 2011, pp. 283-294, doi: 10.1145/1993498.1993532.

[5] B. Roziere, J. M. Zhang, F. Charton, M. Harman, G. Synnaeve, and G. Lample, "Leveraging automated unit tests for unsupervised code translation," arXiv:2110.06773, 2021, revised 2022. [Online]. Available: https://arxiv.org/abs/2110.06773.

[6] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374.

[7] Python Software Foundation, "decimal - Decimal fixed-point and floating-point arithmetic," Python 3.12 Documentation. [Online]. Available: https://docs.python.org/3.12/library/decimal.html. Accessed: Oct. 10, 2026.

[8] Docker, Inc., "Resource constraints," Docker Documentation. [Online]. Available: https://docs.docker.com/engine/containers/resource_constraints/. Accessed: Oct. 10, 2026.

[9] Ollama, "Generate a chat message," Ollama Documentation. [Online]. Available: https://docs.ollama.com/api/chat. Accessed: Oct. 10, 2026.